Hands-on Testing Workshop: FastAPI Cars CRUD

Build confidence with the **three testing tiers**: unit, integration, and end-to-end (E2E). There is also a separate **API testing exercise**. API tests belong to the integration tier, but have their own section because HTTP behavior is worth learning explicitly.

For **each section**, you will:

1. Copy a complete test into a new file.
2. Run it and see a passing result.
3. Deliberately change an assertion so the test fails.
4. Restore the assertion, then write a new test yourself from a specification.

You do not need to modify the app to complete these exercises. The project already contains additional example tests, so your new files can live alongside them.

1. Set up and run the app

Prerequisites

- Python **3.11 or newer**.
- A terminal and a text editor (VS Code is fine).
- Internet access for the first installation of Python packages and Playwright's Chromium browser.

Extract `cars-crud.zip` and open a terminal in the extracted **`cars-crud/` directory**, where `requirements-dev.txt` and `pytest.ini` are located.

macOS / Linux:

```
cd cars-crud
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-dev.txt
```

Windows PowerShell (these commands do not require activating the virtual environment):

```
cd cars-crud
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements-dev.txt
```

For the remaining commands, use `python` after activating the virtual environment on macOS/Linux. On Windows, use `.\.venv\Scripts\python.exe` instead of `python` if the virtual environment is not active.

Start the application

Run this in **Terminal A**, from the `cars-crud/` directory:

```
python -m uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

Open:

- App: http://127.0.0.1:8000
- Create car page: http://127.0.0.1:8000/cars/new
- Interactive API docs: http://127.0.0.1:8000/docs

Create a car, view it, edit it, and delete it manually. This familiarizes you with the expected behavior before you automate it. SQLite is initialized automatically: the application stores data in `cars.db`.

Leave Terminal A running while exploring the app. Open a new termina: **Terminal B** in the same project directory for test commands. On macOS/Linux, activate `.venv` there too using `source .venv/bin/activate` (activation is per terminal). On Windows, use `.\.venv\Scripts\python.exe` in place of `python`, as noted above. You can stop the app with **Ctrl+C**. (The automated tests do not require Terminal A to be running.)

Understand the relevant files

```
cars-crud/
|-- app/
|   |-- schemas.py          # Pydantic input validation (unit-test target)
|   |-- database.py         # SQLite operations (integration-test target)
|   |-- main.py             # FastAPI pages and API routes
|   `-- static/             # Raw HTML, CSS and JavaScript (E2E target)
|-- tests/
|   |-- conftest.py         # database_path, client, car_payload fixtures
|   |-- unit/
|   |-- integration/
|   |-- api/
|   `-- e2e/
|       `-- conftest.py     # page and live_server fixtures
|-- requirements-dev.txt
|-- pytest.ini
`-- TESTING_WORKSHOP.md
```

Important: Every test gets its own temporary SQLite database. Tests do **not** modify the real `cars.db`. E2E tests start their own temporary FastAPI server automatically, so you do not have to start a separate server for them.

2. Unit testing: test validation without a database

Goal: Verify a small piece of Python code in isolation. Our target is the `CarInput` schema in `app/schemas.py`. This test does not open SQLite or send HTTP requests.

Step A - Copy a working test

Create `tests/unit/test_workshop_unit.py` and paste:

```
from app.schemas import CarInput


def test_car_text_fields_are_trimmed():
    car = CarInput(
        make="  Mazda  ",
        model="  MX-5  ",
        year=2020,
        color="  Red  ",
    )

    assert car.make == "Mazda"
    assert car.model == "MX-5"
    assert car.color == "Red"
```

Step B - Run it

```
python -m pytest tests/unit/test_workshop_unit.py -v
```

Expected: `1 passed`. This checks that Pydantic strips whitespace from input strings.

Step C - Intentionally make it fail

Change only this line:

```
assert car.make == "Honda"  # deliberately wrong
```

Run the same command again. **Expected:** `1 failed` with an `AssertionError` (actual value `Mazda`, expected `Honda`). Change it back to `"Mazda"` and confirm the test passes again.

Step D - Write a new unit test (your task)

Create a second test called `test_blank_model_is_rejected` in the same file.

Requirements:

- Try to build `CarInput(make="Mazda", model="  ", year=2020)`.
- Assert that a `pydantic.ValidationError` is raised.
- Do not use a database or HTTP requests.
- Your test should pass against the unmodified application.

Hint: `import pytest`, `from pydantic import ValidationError`, and `with pytest.raises(ValidationError): ...`.

Run the file again; you should now see `2 passed`.

3. Integration testing: test Python code with real SQLite

Goal: Verify that application code really saves and retrieves data from a database. The built-in `database_path` fixture gives this test a fresh temporary SQLite file.

Step A - Copy a working test

Create `tests/integration/test_workshop_integration.py` and paste:

```
from app import database
from app.schemas import CarInput


def test_saved_car_can_be_read_from_sqlite(database_path):
    database.initialize(database_path)

    saved = database.create_car(
        database_path,
        CarInput(make="Toyota", model="Yaris", year=2020, color="Blue"),
    )
    loaded = database.get_car(database_path, saved.id)

    assert loaded is not None
    assert loaded.id == saved.id
    assert loaded.make == "Toyota"
    assert loaded.year == 2020
```

Step B - Run it

```
python -m pytest tests/integration/test_workshop_integration.py -v
```

Expected: `1 passed`. `create_car()` and `get_car()` use separate SQLite connections, so the test also checks that the write was committed and persisted.

Step C - Intentionally make it fail

Change:

```
assert loaded.year == 2021  # deliberately wrong
```

Run the test again. **Expected:** `1 failed` (the stored year is `2020`). Restore `2020` and rerun to see a pass.

Step D - Write a new integration test (your task)

Create `test_deleting_one_car_keeps_the_other` in the same file.

Requirements:

- Initialize a fresh SQLite database using `database.initialize(database_path)`.
- Create two different cars using `database.create_car(...)` and `CarInput(...)`.
- Delete **only the first** car using `database.delete_car(database_path, first.id)`.
- Assert the deleted car cannot be found with `database.get_car(...)`.
- Assert the second car is still present and `database.list_cars(database_path)` contains exactly that one car.
- Your test should pass against the unmodified application.

Hint: The existing `test_saved_car_can_be_read_from_sqlite` shows how to create a car. The database functions are in `app/database.py`.

Run the file again; you should see `2 passed`.

4. API testing: test the HTTP contract

API tests are a **kind of integration test**. Here they run requests against the FastAPI app using `TestClient`, without starting Uvicorn or opening a browser. The `client` fixture creates a real, isolated SQLite database.

Step A - Copy a working test

Create `tests/api/test_workshop_api.py` and paste:

```
def test_post_car_returns_201_and_json(client):
    payload = {
        "make": "Ford",
        "model": "Fiesta",
        "year": 2018,
        "color": "Black",
    }

    response = client.post("/api/cars", json=payload)

    assert response.status_code == 201
    car = response.json()
    assert car["id"] > 0
    assert car["make"] == "Ford"
    assert car["model"] == "Fiesta"
    assert response.headers["location"] == f"/api/cars/{car['id']}"
```

Step B - Run it

```
python -m pytest tests/api/test_workshop_api.py -v
```

Expected: `1 passed`. You verified the HTTP status, JSON response, generated ID, and `Location` header.

Step C - Intentionally make it fail

Change:

```
assert response.status_code == 200  # deliberately wrong
```

Rerun. **Expected:** `1 failed` (the API correctly returns `201 Created`). Restore `201`.

Step D - Write a new API test (your task)

Create **`test_invalid_year_is_not_saved`** in the same file.

Requirements:

- Send `POST /api/cars` with a year of `1800` and otherwise valid car data.
- Assert the response status is **`422`**.
- Send `GET /api/cars` afterward and assert it returns an empty list.
- Do not use a browser or call database functions directly.

Hint: You can reuse `client.post(..., json=...)` and `client.get(...)`.

Run the file again; you should see **`2 passed`**.

5. End-to-end (E2E) testing: test in a real browser

Goal: Test the complete journey: browser -> HTML form -> JavaScript `fetch()` -> FastAPI -> SQLite -> browser display.

Step A - Install Chromium once

```
python -m playwright install chromium
```

On Linux, if system libraries are missing, install those dependencies using the documented Playwright command:

```
python -m playwright install --with-deps chromium
```

No `pytest-playwright` plugin is required; the project already supplies `page` and `live_server` fixtures in `tests/e2e/conftest.py`.

Step B - Copy a working test

Create **`tests/e2e/test_workshop_e2e.py`** and paste:

```
import re

from playwright.sync_api import expect


def test_user_creates_car_in_browser(page, live_server):
    page.goto(f"{live_server}/cars/new")

    page.get_by_label("Make", exact=True).fill("Toyota")
    page.get_by_label("Model", exact=True).fill("Yaris")
    page.get_by_label("Year", exact=True).fill("2020")
    page.get_by_label("Color", exact=False).fill("Blue")
    page.get_by_role("button", name="Create car").click()

    expect(page).to_have_url(re.compile(r"/cars/\d+$"))
    expect(page.get_by_role("heading", name="Toyota
Yaris")).to_be_visible()
```

Step C - Run it

```
python -m pytest tests/e2e/test_workshop_e2e.py -v
```

Expected on a local computer with Chromium working: `1 passed`. You can watch the browser with:

```
python -m pytest tests/e2e/test_workshop_e2e.py -v --headed
```

There is no need to launch Uvicorn manually: the `live_server` fixture starts and stops a server on a free local port for each test.

Step D - Intentionally make it fail

Change the final assertion to:

```
expect(page.get_by_role("heading", name="Toyota Corolla")).to_be_visible()
```

Rerun. **Expected:** `1 failed` because the screen says `Toyota Yaris`, not `Toyota Corolla`. Playwright waits up to its default assertion timeout for the wrong heading, then reports the failure. Restore `Toyota Yaris`.

Step E - Write a new E2E test (your task)

Create **`test_user_edits_car_and_change_survives_reload`** in the same file.

Requirements:

- Open `/cars/new` and create a **Honda Civic**, year **2022**.
- After creation, click the **Edit car** link.
- Change the model to **Accord** and the year to **2023**.
- Click **Save changes**.
- Assert the detail page has the heading **Honda Accord**.
- Reload the page and assert the model still displays **Accord** and the year **2023**.
- Interact only through Playwright browser actions; do not call Python database functions or `TestClient`.

Hints: Use `page.get_by_role("link", name="Edit car")`, `page.get_by_label(...)`, `page.reload()`, and `expect(...)`. The detail page uses `#car-model` and `#car-year` for display values. Check `tests/e2e/test_browser.py` if you need an example.

Run the file again; you should see `2 passed` on a machine where Chromium can access the local server.

6. Run everything and inspect failures

After completing the exercises:

```
# Only your four workshop files (expected: 8 tests)
python -m pytest \
  tests/unit/test_workshop_unit.py \
  tests/integration/test_workshop_integration.py \
  tests/api/test_workshop_api.py \
  tests/e2e/test_workshop_e2e.py -v

# All tests (existing examples plus your new exercises)
python -m pytest -q
```

The multiline command above is for macOS/Linux shells. On Windows PowerShell, use one line:

```
python -m pytest tests/unit/test_workshop_unit.py
tests/integration/test_workshop_integration.py
tests/api/test_workshop_api.py tests/e2e/test_workshop_e2e.py -v
```

To run tests individually:

```
python -m pytest tests/unit -q
python -m pytest tests/integration -q
python -m pytest tests/api -q
python -m pytest tests/e2e -q
```

How to read outcomes:

- `PASSED`: the code satisfied the test's assertions.
- `FAILED`: the test ran, but an assertion did not match; inspect actual versus expected values.

- `ERROR`: the test could not be set up or run (for example, missing browser libraries, import errors, or a server startup problem).

When a test fails, first identify **which layer** you are testing, then read the final assertion/error message, not just the entire traceback. During the intentional-failure steps, a `FAILED` result is the **desired observation**. After restoring the assertion, the same test should return to `PASSED`.

7. Completion checklist

- [ ] I can start the app, create a car, edit it, and delete it manually.
- [ ] My copied **unit** test passes, fails when I break the assertion, and passes when restored.
- [ ] My own unit test rejects a blank model.
- [ ] My copied **integration** test passes, fails, and passes again.
- [ ] My own integration test deletes one car without affecting another.
- [ ] My copied **API** test passes, fails, and passes again.
- [ ] My own API test rejects an invalid year without saving data.
- [ ] My copied **E2E** test passes, fails, and passes again in a real browser.
- [ ] My own E2E test edits a car and verifies the change survives a reload.
- [ ] I can explain what each tier catches and why the tests do not touch the real database.